RAKSHA - Real-time AI Knowledge Screening & Hazard Audit

Version 1.5

YP-Raksha

TECHNICAL MANUAL

YP-Raksha Technical Documentation

Architecture, APIs, data model, security controls and Browser AI implementation

Contents and document index

Section page numbers include the cover page. Headings are listed in reading order.

System overview 4
Technology stack 4
Repository layout 5
Core module 5
- Providers 5
- Schemas 5
- MCP client 5
- Key selection and networking 6

Framework module 6
- ConfigStore 6
- LogsStore 7
- RBAC 7
- Other framework packages 8

Plugins 8
HTTP surface 9
- Entry point and flags 9
- Middleware assembly 9
- Request identification on inference 9
- Route groups 10
- Health and observability endpoints 11

Data model key tables 11
- ConfigStore tables 11
- Browser AI tables 11

Browser AI Guard technical detail 12
- Backend routes 12
- Guard fleet authentication no user session 13
- Guard desktop agent 13
- mitmproxy addon 14
- Guard distribution and hot update 15

Frontend 15
- Build and integration 15
- Pages 15
- Sidebar and section gating 16
- Browser AI UI implementation files 16

Security model 16
Build test and developer workflow 17
- Gateway 17
- UI 17
- Guard 17
- Docker Helm 18
- Conventions 18

Extension points 18
Glossary 19
Enterprise deployment context 19
Implementation boundaries relevant to product selection 19
Operational security review points 20
Product architecture and request flows 20
- Inference request lifecycle 21
- Browser AI Guard decision lifecycle 22

Administration UI and policy change lifecycle 23
Optional MCP tool lifecycle 24
State scaling and component ownership 24
Error handling retries and recovery 25
Error response contract 25
Common failure classes and caller action 25
Retry key rotation and fallback behavior 26
Streaming WebSocket and partial response errors 27
Operational recovery matrix 27
Incident evidence and escalation 28
Deployment prerequisites and configuration 28
Required platform dependencies 28
Runtime configuration groups 28
Production packaging cautions 29
Installation upgrade and rollback runbook 29
Day 2 operations availability and data protection 30
Health and monitoring 30
Backup restore and DR dependencies 30
Change control and key rotation 30
Capacity qualification 31
Technical acceptance checklist 31
Source and maintenance note 31

Audience: developers, architects, integrators

Scope: architecture, modules, data model, HTTP surface, plugins, Browser AI internals, UI, security model

Baseline: repository workspace revision `d90b0359`, reviewed 24 October 2026

System overview

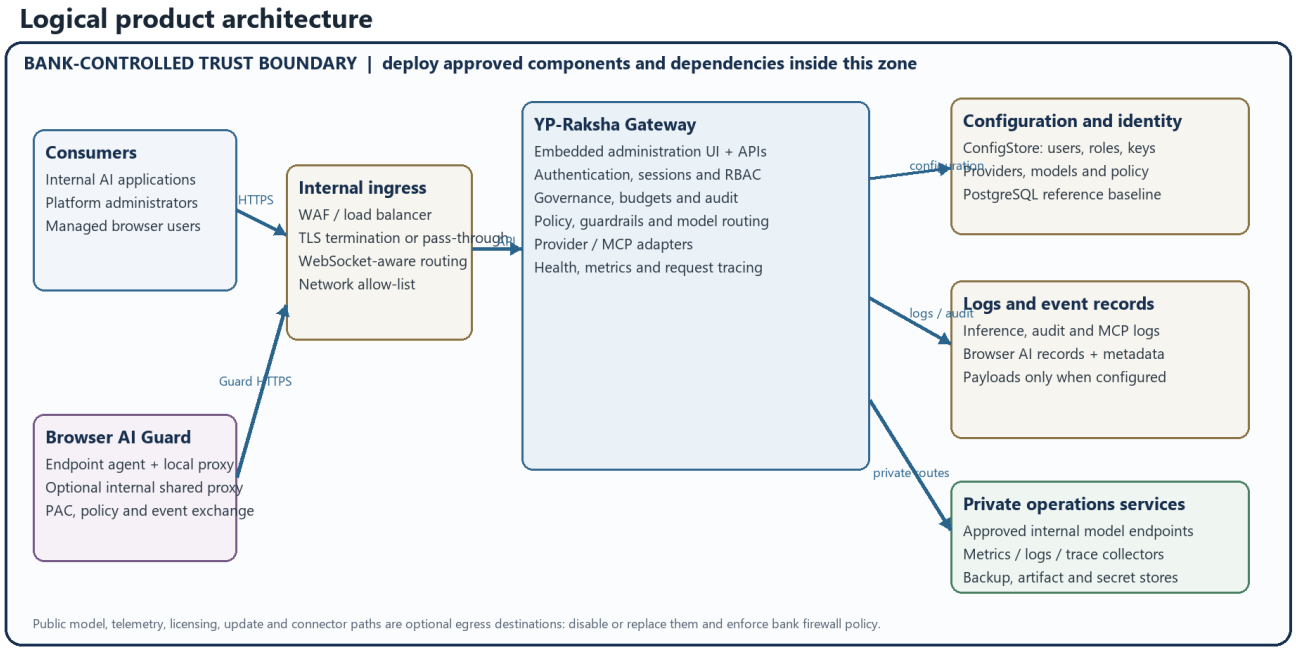


Figure 1. Logical YP-Raksha architecture. Optional services and destinations depend on configuration; the bank must enforce the residency boundary.

Two data planes exist in the same binary:

1. **Data plane** (`/v1/*`, pass-through integrations) — request/response proxying to AI providers, with governance, guardrails, caching, logging, tracing.

2. **Control plane** (`/api/*`) — the dashboard: configuration, RBAC, observability and the whole Browser AI guardrail control plane.

Technology stack

Layer	Technology	Source of truth
Gateway	Go 1.26.4, `valyala/fasthttp`, `fasthttp/router`, `automaxprocs`	`go.work`, `transports/gateway-http/main.go`
Persistence	GORM; Postgres / MySQL / SQLite (CGO `go-sqlite3`, `sqlite_static`)	`framework/configstore`, `framework/logstore`
Provider SDKs	Vendor HTTP clients + official SDKs	`core/providers/*`
Plugins	Go interfaces, optional `.so` dynamic loading (`SharedObjectPluginLoader`)	`plugins/`, `server/plugins.go`
MCP	Model Context Protocol client/server, OAuth2 sessions	`core/mcp`, `handlers/mcp*.go`

Layer	Technology	Source of truth
Frontend	React 19, TypeScript 5.9, Vite 8, TanStack Router/Table, Redux Toolkit, Radix UI, Tailwind 4, Monaco, Recharts, Zod	`ui/package.json`
Guard agent	Python 3.11+, mitmproxy addon, PyInstaller, Inno Setup (Windows), pkg/dmg scripting (macOS)	`apps/browser-guard/**`
Deploy	Docker (multi-stage), docker compose, Helm (Postgres, Redis, Qdrant, Weaviate, HPA, Ingress)	`deploy/**`, `docker-compose.yml`
Tooling	Make targets, gotestsum, Playwright, newman, oxlint/oxfmt	`Makefile`

Repository layout

```
core/                        Go module: the AI engine
  providers/<vendor>/        one package per provider (chat, responses, images, audio, video, batch…)
  schemas/                   shared types (requests, responses, plugin interfaces, errors)
  mcp/                       MCP client, code mode, credential store
  keyselectors/              weighted/random key selection strategies
  network/                   HTTP client utilities (retries, timeouts, proxies)

framework/                   Shared infrastructure
  configstore/               ConfigStore + WorkspaceStore; tables/ has every GORM model
  logstore/                  LLM/MCP/Browser AI log storage, aggregations, matviews, cleaners
  rbac/                      Resource×Operation enforcement, route → requirement mapping
  encrypt/                   AES encryption at rest for secrets in DB
  cluster/, loadbalancer/, routing/, circuitbreaker/, kvstore/, featureflags/
  connectors/, objectstore/, vectorstore/, oauth2/, mailer/, alerts/, tracing/
  migrator/, mysqlconn/, postgresconn/    DB drivers + schema migration helpers

plugins/                     telemetry, prompts, logging, governance, otel, semanticcache,
                             compat, maxim, guardrails, connectors, modelcatalogresolver,
                             mocker, jsonparser

transports/gateway-http/      The server binary
  main.go                    flags, logger, version banner, bootstrap/start
  server/server.go           GatewayHTTPServer: bootstrap, middleware assembly, graceful shutdown
  server/plugins.go          built-in plugin instantiation + ordering
  handlers/*.go              every /api and /v1 handler (one file per feature area)
  integrations/*.go          pass-through provider surfaces
  websocket/, ui/            WS pool/sessions; embedded UI assets

ui/                          React dashboard (Vite). pages under ui/app/workspace/**
apps/browser-guard/          Python Guard agent + mitmproxy addon + installers + release artifacts
configs/                     config.json, mcp-library.json, model-parameters.json, pricing.json
deploy/                      docker/, helm/gateway/
docs/                        this documentation set
```

Core module

Providers

Each directory under `core/providers/<vendor>` implements the shared provider contract from `core/schemas`. A provider package contains files such as `chat.go`, `responses.go`, `images.go`, `speech.go`, `transcription.go`, `videos.go`, `batch.go`, `models.go`, `errors.go`, `types.go`, `utils.go`.

Capability differences that matter (from the files present):

Provider(s)	Notable capability files
`openai`	chat, responses, responseslifecycle, text, images, speech, transcription, videos, batch, files, embedding, realtime, websocket, large_payload, passthrough_usage

Provider(s)	Notable capability files
anthropic, gemini	chat, responses, count_tokens / models, embedding (gemini), types
vertex	chat, batch, cachedcontents, count_tokens, embedding, rerank, models
bedrock, bedrockmantle	chat, responses, embedding, images, batch
replicate, runware, runway	chat, images, videos, files
mistral	adds ocr.go on top of chat/transcription/cachedcontents
elevenlabs	audio-only
ollama, sgl, vllm, openrouter, parasail, opencode, groq, cerebras, deepseek, fireworks, huggingface, nebius, perplexity, xai, cohere	vendor subsets (mostly OpenAI-compatible shapes)
openaicompat	generic OpenAI-compatible provider + registry.go for many compatible endpoints
azure	Azure OpenAI, deployment-aware

Shared helpers live in core/providers/utils: decompression.go, sse.go, stream.go, streamterminaldetector.go, pagination.go, images.go, audio.go, file.go, videos.go, bodysigner.go (request body signing for AWS-style auth), modelparamscache.go, largeresponse.go, passthrough_stream.go, fetch.go, models.go, utils.go.

Schemas

core/schemas holds the types shared by the gateway and every plugin: chat / responses / text / embedding / image / audio / video request-response types, GatewayConfig, GatewayContext, ProviderConfig, NetworkConfig, the plugin interfaces (BasePlugin, LLMPlugin, MCPPlugin, ObservabilityPlugin, HTTPTransportPlugin, GatewayHTTPMiddleware), error types and log level/output enums.

MCP client

core/mcp implements the Model Context Protocol client used by the MCP Gateway: connection lifecycle, tool discovery, tool execution, per-user credentials (credstore), "code mode" (codemode) and utilities. The HTTP layer wraps it in handlers/mcp.go (clients, library, tool groups), handlers/mcpserver.go, handlers/mcpsessions.go and the OAuth2 set (mcpoauth2.go, mcpoauth2discovery.go, mcpoauth2issuance.go, mcpoauth2consent.go, mcpoauth2sessions.go, mcpoauth2jwt.go).

Key selection and networking

- core/keyselectors — weighted/random selection across multiple keys per provider.
- core/network — the outbound HTTP client for provider calls (proxy, TLS, timeouts, retries),

driven by each provider's NetworkConfig.

Framework module

ConfigStore

framework/configstore is the configuration database layer.

- Drivers: postgres.go, mysql.go, sqlite.go, rdb.go; interface in store.go; errors in errors.go.
- migrations.go plus framework/migrator (addcolumn.go, dropcolumn.go) handle schema evolution.
- tables/ holds one file per domain: user.go, workspace.go, enterprise.go, team.go,

customer.go, virtualkey.go, key.go, provider.go, model.go, modelconfig.go, modelpricing.go, modelparameters.go, pricingoverride.go, budget.go, ratelimit.go, routingrules.go, mcp*.go, mcpoauth2*.go, prompts.go, promptVersions.go, promptSessions.go, skills.go, folders.go, featureflag.go, plugin.go, config.go, clientconfig.go, env.go, sessions.go, temptokens.go, dlock.go (distributed locks), logstore.go, vectorstore.go, smtp.go, confighash.go.

- workspace.go seeds the **RBAC catalog**, the three **system roles**, and exposes the resource/operation/name catalogs used by backend and UI.
- Secrets in the DB are encrypted with framework/encrypt when GATEWAY_ENCRYPTION_KEY is set (tables/encryption.go, configstore/encryption.go).

LogsStore

framework/logstore stores and aggregates request logs.

- Files: logger.go, store.go, rdb.go, postgres.go, mysql.go, sqlite.go, hybrid.go.
- browser_ai.go is the **entire Browser AI persistence layer** (see §9).
- Aggregations for dashboard/log pages: matviews.go (materialized views), tables.go, cleaner.go (retention), asyncjob.go (async inference jobs), payload.go.

RBAC

1. **Resource × Operation** — framework/rbac/enforce.go, PathRequirementFor(method, path) returns the requirement for a route (or nil = not RBAC-gated). admin resolves to *allow all*; an unknown/unseeded role yields an **empty set = deny**. View and Read are interchangeable.

2. **Sidebar section grants** — framework/rbac/sections.go, SectionRequirementFor(method, path), used for section-scoped sessions. Keys are parent or parent/child; parent/* accepts the parent or any child. Legacy observability/browser-ai is normalised to browser-ai.

Resource catalog (33): `GuardrailsConfig, GuardrailsProviders, GuardrailRules, UserProvisioning, Cluster, Settings, Users, Logs, Observability, Dashboard, VirtualKeys, ModelProvider, Plugins, MCPGateway, MCPToolGroups, MCPLogs, AdaptiveRouter, AuditLogs, Customers, Teams, RBAC, Governance, RoutingRules, PromptRepository, PromptDeploymentStrategy, SkillsRepository, AccessProfiles, APIKeys, Inference, Metrics, FeatureFlags, CircuitBreaker`

Operation catalog (6): Read, View, Create, Update, Delete, Download

System roles (auto-seeded in configstore/workspace.go, then merged on every boot so new permissions are picked up — except for roles a customer has customised):

Role	DAC	Grants
admin	all-data	every permission
sub_admin	all-data	Read/View on Dashboard, Inference, Observability, MCPLogs, RoutingRules, Metrics; **full** (Read/View/Create/Update/Delete/Download) on Logs — this is what gates all Browser AI write APIs; full on VirtualKeys, PromptRepository; View/Read/Update on Governance; View/Read/Create/Update on MCPGateway, MCPToolGroups; View/Read on ModelProvider. **No** master provider keys, **no** system settings
user	own-data	Read/View **only**, on Dashboard, Logs, Inference, PromptRepository, Observability, MCPGateway (the readIDs set). The UI labels this "Prompt Repository only"; the backend also allows read-only views of logs/observability/MCP. Use allowed_sections to narrow what a user actually sees

Custom roles are created in the UI. admin is always re-synced to the full catalog.

Browser AI note: every `/api/browser-ai/*` route maps to the **`Logs`** RBAC resource (`PathRequirementFor`), so a role needs `Logs: View/Update` (or the `browser-ai` section grant) to see the Browser AI pages.

Other framework packages

Package	Purpose	API surface
`cluster`	multi-node config propagation	`/api/cluster`, `/internal/cluster/kv`
`loadbalancer`	adaptive routing across providers/keys	`/api/load-balancer`, `/api/load-balancer/routes`
`routing`	routing-rule engine	`/api/governance/routing-rules`, `/api/routing-rules`
`circuitbreaker`	failure isolation policies	`/api/circuit-breaker/policies`, `/state`
`featureflags`	server-side flags	`/api/feature-flags`
`rbac`, `oauth2`, `temptoken`	permissions, OAuth2 provider, short-lived tokens	`/api/rbac/*`, `/api/scim/oauth/*`
`mailer`, `alerts`	SMTP and alert delivery (webhooks)	`/api/smtp-config`, `/api/alert-channels`
`connectors`	external sinks (BigQuery, Kafka, Datadog, New Relic, PubSub)	`/api/connectors`
`objectstore`, `vectorstore`, `kvstore`	blob / vector / KV backends (Qdrant, Weaviate, Redis)	`/api/vector-store-config`, `/api/cache`
`tracing`	trace store + spans	`/api/logs` trace views
`queryscope`, `mcpcatalog`, `mcptoolgroups`, `mcp_headers`, `modelcatalog`, `encrypt`, `envutils`	supporting services	–
`streaming`	SSE/stream utilities shared by transports	–

Plugins

A plugin implements one or more core interfaces and is loaded either as a **built-in** (by name) or as a **dynamic `.so`** via `SharedObjectPluginLoader` (`server/plugins.go`).

Built-in load order (fixed placement indexes):

#	Plugin	Role
1	`telemetry`	request metrics/counters
2	`prompts`	prompt repository integration
3	`logging`	LLM/MCP log persistence
4	`governance`	virtual keys, budgets, rate limits, teams, customers, business units
5	`otel`	OpenTelemetry export
6	`semanticcache`	embedding-based response caching
7	`compat`	parameter compatibility/conversion between client and provider shapes
8	`maxim`	Maxim observability integration

#	Plugin	Role
—	guardrails	rule-based input/output validation + provider config
—	connectors	external log/analytics sinks
—	modelcatalogresolver	resolves model metadata/pricing
—	mocker	mock provider for tests/E2E
—	jsonparser	tolerant JSON parsing helpers

Plugin config lives in `config.json` under `plugins`; plugins can be enabled/disabled and reordered at runtime (`GET`/`POST /api/plugins`). A plugin that fails to load is marked disabled and reported in the startup plugin-status table (`plugin status: <name> - <status>`).

HTTP surface

Entry point and flags

`main.go`:

Flag	Env fallback	Default
`-port`	`APP_PORT`	`8001`
`-host`	`APP_HOST` / `APP_HOST`	`localhost`
`-app-dir`	`APP_DIR`	`./data`
`-log-level`	`LOG_LEVEL`	`info`
`-log-style`	`LOG_STYLE`	`json`

Startup prints an ASCII banner (`-ldflags -X main.Version=…`), starts the pprof server, configures the logger, then `Bootstrap()` → `Start()`.

Middleware assembly

Server handler wrapper (`server.go`):

```
SecurityHeadersMiddleware → CORS → RequestDecompression → router
```

API middleware chain (`apiMiddlewares`, applied to every `/api/*` route):

```
AuthMiddleware.APIMiddleware()      session cookie / Bearer session token
WorkspaceAuditMiddleware            audits mutating /api/* into Audit Logs
RBACMiddleware                      resource × operation + section grants
```

Inference middleware chain (`/v1/*` and pass-through):

```
TracingMiddleware → TransportInterceptor → AuthMiddleware.InferenceMiddleware → handler
```

`TransportInterceptor` runs `HTTPTransportPostHook` plugins and captures their log entries; the tracing defer runs afterwards so plugin logs land inside the trace.

Request identification on inference

- `Authorization: Bearer <virtual-key|provider-key>` or `x-api-key: <key>`.
- Dashboard-originated inference is enriched from the session cookie

(`enrichInferenceFromDashboardSession`); provider headers are normalised to a single `Authorization: Bearer …` (`x-api-key` is deleted).

- Password-as-Bearer (base64 `user:pass`) is **removed** — only session tokens are accepted as Bearer.
- Virtual Key self-service quota: `GET /api/governance/virtual-keys/quota` (the VK header is the credential).

Route groups

Group	Representative routes	Handler file(s)
Health / UI	`GET /health`, `GET /robots.txt`, / (SPA)	`health.go`, `ui.go`
Session / auth	`/api/session/login`, `/logout`, `/register`, `/register/verify`, `/register/resend`, `/forgot-password`, `/verify-otp`, `/reset-password`, `/forgot-username`, `/is-auth-enabled`, `/ws-ticket`, `/api/session/users*`	`session.go`, `session_register_verify.go`, `auth_security.go`
Workspace / RBAC	`/api/users/{id}/role`, `/api/roles*`, `/api/permissions`, `/api/resources`, `/api/operations`, `/api/rbac/*`, `/api/access-profiles*`	`workspace.go`, `rbac.go`, `rbac_enforce.go`, `accessprofiles.go`
Governance	`/api/governance/virtual-keys*`, `/customers*`, `/teams*`, `/business-units*`, `/routing-rules*`, `/complexity-analyzer-config`, `/api/guardrails/*`	`governance.go`, `businessunits.go`, `team_members.go`, `guardrails.go`
Inference	`/v1/chat/completions`, `/v1/completions`, `/v1/responses*`, `/v1/embeddings`, `/v1/rerank`, `/v1/ocr`, `/v1/audio/*`, `/v1/images/*`, `/v1/videos*`, `/v1/batches*`, `/v1/files*`, `/v1/containers*`, `/v1/realtime`, `/v1/async/*`	`inference.go`, `asyncinference.go`, `realtime_*.go`, `webrtc_realtime.go`
Pass-through	`/openai/*`, `/anthropic/*`, `/genai/*`, `/bedrock/*`, `/cohere/*`, `/cursor/*`, `/litellm/*`, `/langchain/*`, `/pydanticai/*`	`integrations/*.go`
Observability	`/api/logs*`, `/api/logs/dashboard`, `/api/logs/filterdata`, `/api/logs/rankings*`, `/api/logs/histogram/*`, `/api/mcp-logs*`, `/api/audit-logs`, `/api/connectors`, `/api/alert-channels`	`logging.go`, `auditlogs.go`, `alertchannels.go`, `integrations.go`
Providers / models	`/api/providers*`, `/api/keys*`, `/api/models*`, `/api/model-configs*`	`providers.go`, `provider_keys.go`, `list_models_vk.go`
MCP	`/api/mcp/clients`, `/api/mcp/library*`, `/api/mcp/client*`, `/api/mcp/tool-groups*`, `/api/mcp-logs`, `/api/mcp/inference`	`mcp.go`, `mcpserver.go`, `mcpsessions.go`, `mcptoolgroups.go`, `mcpheaders.go`
Prompts / skills	`/api/prompt-repo*`, `/api/prompt-deployments`, `/api/skills*`, `/api/skills/serve/*`	`prompts.go`, `prompts_lifecycle.go`, `skills.go`, `skills_serving.go`
Config / platform	`/api/config`, `/api/proxy-config`, `/api/vector-store-config`, `/api/smtp-config`, `/api/cache`, `/api/plugins`, `/api/feature-flags`, `/api/cluster`, `/api/circuit-breaker/*`, `/api/load-balancer*`	`config.go`, `plugins.go`, `featureflags.go`, `cluster.go`, `circuitbreaker.go`, `cache.go`
Enterprise / SCIM	`/api/scim/*`, `/api/scim/oauth/*`	`scim.go`, `scim_handlers.go`, `scim_oauth.go`, `scim_auth.go`
Browser AI	see §9	`browser_ai*.go`
WebSocket	`/ws`, WS tickets, responses/realtime WS	`websocket.go`, `wsrealtime.go`, `wsresponses.go`, `ws_ticket.go`
Dev	`/api/dev/*` + pprof (dev mode only)	`devpprof*.go`

Health and observability endpoints

- GET /health and GET /api/health ping configured ConfigStore, LogsStore and VectorStore services concurrently with a 10-second timeout. A healthy response reports status healthy and component checks; a dependency failure returns HTTP 503. The client.disable_db_pings_in_health option can disable these dependency pings, so a process-only health response must not be used as full readiness.
- `GET /robots.txt` — static `Allow: /`.
- `GET /api/version` — build version.
- Logging skip list (no request logging): `/health`, `/_next`, `/api/dev`.
- Metrics (Prometheus) and traces come from the telemetry/otel plugins and the Logs API.
- Dev builds additionally expose pprof handlers (gated by `devpprof_prod.go` in release builds).

Data model key tables

ConfigStore tables

- **`governance_users`** — `id, username(uniq), email, password, role(admin|sub_admin|user), status (pending|approved|rejected|email_unverified|disabled), budget, rate_limit, budget_id, rate_limit_id, allowed_prompt_repos, allowed_sections, reviewed_at, external_id, created_at, updated_at. IsApproved()` treats an empty status as approved (legacy rows).
- **`rbac_roles`** — `id, name(uniq), description, is_system_role, dac(all-data|own-data), permission_ids(JSON), created_at, updated_at`.
- **`access_profiles`** — `id, name(uniq), description, is_active, version, calendar_aligned, tags(JSON), spec(JSON), created_at, updated_at` (reusable governance policy templates with activate / deactivate / clone operations).
- **`governance_business_units`** — `id, name(uniq), team_ids(JSON), budget(JSON), rate_limit(JSON)`.
- Grouped in `tables/workspace.go`: `alert_channels`, `circuit_breaker_policies`, `mcp_tool_groups`, `prompt_deployments`, `workspace_settings`, `virtual_key_users`, `virtual_key_teams`, `virtual_key_customers`, `audit_logs`.
- Plus the core governance tables: teams, customers, virtual keys, provider keys, models, model configs, pricing + pricing overrides, budgets, rate limits, routing rules, prompts/versions/sessions, skills, folders, feature flags, plugins, sessions, temp tokens, distributed locks, SMTP, vector-store config, MCP clients/library/OAuth2 issuance.

Browser AI tables

Table	Purpose	Notable columns
`browser_ai_logs`	one row per intercepted prompt / upload	`id, timestamp, platform, domain(host), user_prompt_full, action, status, rule_triggered, risk_score, predictive_risk, agent_id, agent_hostname, client_ip, reply_bot_text, attachment_name/type/size/path/expiry, metadata(JSON)`
`browser_ai_search_logs`	search-engine queries + clicked results	`id, timestamp, engine, browser, is_incognito, query, clicked_url, clicked_title, url, host, agent_id, predictive_risk`

Table	Purpose	Notable columns
browser_guard_rules	guard rules	id, name, pattern, rule_type(regex or ai_bot), severity, action, warning_message, active, description, bot_provider, bot_model, bot_prompt, bot_reference_image, bot_reference_image_type
browser_ai_target_websites	monitored AI sites	id, domain(uniq), platform_name, monitored, block_entire_website, host_role(ui/chat/file), status(MONITORED/PAUSED/BLOCKED), parent_id
browser_ai_control_settings	single-row interaction policy (browser-controls-default)	upload/download warning text and policy toggles
browser_ai_agents	Guard fleet inventory	id, hostname, agent_type(endpoint/network), status(active/uninstall_pending/uninstalled), last_seen_at, health_detail, contact_email(+pinned), uninstall_key_hash, uninstall_key_enc, has_uninstall_key, uninstall_key_rotated_at, proxy_bundle_sha
browser_ai_agent_settings	company-level uninstall key	id, uninstall_key_enc
browser_guard_fleet_config	company Guard defaults that agents pull on heartbeat	id, … notes
browser_guard_rebuild_logs	Guard package rebuild history	id, status(success/failed), bundle_sha, message, log, created_at
browser_ai_warning_emails	employee warning e-mail log	id, sent_at, to_email, message, agent_id, error_detail

- Attachment bytes stay on disk (APP_DIR/attachments); the DB stores size, path and expiry metadata.
- BrowserAIManager (same file) implements every CRUD/aggregation API used by the handlers,

including GuardSeverityScore (severity → predictive risk score + label) and NormalizeBrowserAIAgentType.

Browser AI Guard technical detail

Backend routes

Logs / insights

```
GET    /api/browser-ai/logs                 list + filter prompt logs
GET    /api/browser-ai/logs/stats           aggregated stats
DELETE /api/browser-ai/logs                 delete all
POST   /api/browser-ai/logs/bulk-delete
DELETE /api/browser-ai/logs/{id}
GET    /api/browser-ai/search-logs
POST   /api/browser-ai/search-logs          agent ingest
DELETE /api/browser-ai/search-logs
POST   /api/browser-ai/search-logs/bulk-delete
DELETE /api/browser-ai/search-logs/{id}
GET    /api/browser-ai/insights/stats       Guard Insights
```

Rules / controls / targets

```
GET|POST       /api/browser-ai/rules            + POST /rules/import, PUT|DELETE /rules/{id}
POST           /api/browser-ai/rules/test-bot          test an AI Guard Bot rule
POST           /api/browser-ai/rules/generate-regex    generate regex from a policy description
GET|PUT        /api/browser-ai/controls                interaction controls (single row)
GET            /api/browser-ai/targets                 + POST /targets, POST /targets/import,
```

```
                                                    PUT|DELETE /targets/{id}
```

Fleet / agents / setup

```
GET    /api/browser-ai/agents                       list agents (filters: status, source, search)
POST   /api/browser-ai/agents/heartbeat             agent registration + health (Guard secret)
POST   /api/browser-ai/agents/bulk-delete
DELETE /api/browser-ai/agents/{id}
PUT    /api/browser-ai/agents/{id}/contact-email
GET    /api/browser-ai/agents/settings
GET|PUT /api/browser-ai/agents/uninstall-key        company uninstall key
GET    /api/browser-ai/agents/{id}/uninstall-key
POST   /api/browser-ai/agents/{id}/uninstall-key/rotate
POST   /api/browser-ai/agents/uninstall-verify      employee key check
POST   /api/browser-ai/agents/uninstall             agent requests removal
POST   /api/browser-ai/agents/uninstall-ack
POST   /api/browser-ai/agents/uninstall-status
POST   /api/browser-ai/agents/{id}/remote-uninstall  admin-initiated removal
GET|PUT /api/browser-ai/fleet-config                company-wide Guard defaults
GET    /api/browser-ai/setup/info
GET    /api/browser-ai/setup/download.zip           aliases: -windows.zip, -mac.zip
POST   /api/browser-ai/setup/rebuild                rebuild + republish Guard packages
GET    /api/browser-ai/setup/rebuild-history
GET    /api/browser-ai/setup/proxy-bundle.json
GET    /api/browser-ai/setup/proxy-bundle.zip       hot-update bundle for installed Guards
POST   /api/browser-ai/send-warning-email
```

Ingest / PAC / helpers

```
POST /api/browser-ai/intercept           agent prompt/upload event
POST /api/browser-ai/intercept-file      agent file event
GET  /api/browser-ai/attachments/{id}    attachment download (auth-gated)
GET  /api/browser-ai/proxy.pac           PAC script
GET  /api/browser-ai/pac                 PAC script (alias used by GPO / network proxy)
GET  /api/browser-ai/ollama-models       models available for AI Guard Bot rules
```

Guard fleet authentication no user session

`isPublicBrowserAIRoute` and `isGuardKeyBrowserAIRoute` (`handlers/middlewares.go`) let the Guard agent call the backend **without a dashboard session cookie**:

- **Public (secret-validated):** `intercept`, `intercept-file`, `search-logs` (POST), `proxy.pac`, `pac`, `agents/heartbeat`, `agents/uninstall-verify`, `agents/uninstall`, `agents/uninstall-ack`, `agents/uninstall-status`.
- **Guard-key:** `targets`, `rules`, `controls`, `fleet-config` (GET), `setup/download*.zip`, `setup/proxy-bundle.json`, `setup/proxy-bundle.zip`.
- The fleet secret is `GATEWAY_GUARD_SECRET`. With `GATEWAY_GUARD_REQUIRE_SECRET=1` (production default in `.env.example` and compose) agent APIs **fail closed** when the secret is missing or wrong (`browser_ai_guard_security.go`).
- `GATEWAY_PAC_ALLOW_QUERY_PROXY=0` (default) makes the PAC ignore a client-supplied `?proxy=` and use only fleet/env values — anti PAC-hijack.

Guard desktop agent

`gateway_agent.py` is the entry point. Supporting modules:

Module	Responsibility
`guard_bootstrap.py`	single-instance lock, logging bootstrap, runtime version resolution

Module	Responsibility
agent_config.py	env → JSON config precedence (GATEWAY_BACKEND_URL, GATEWAY_PROXY_ADDR, GATEWAY_PAC_URL, GATEWAY_PAC_SYNC_SECONDS, GATEWAY_SERVER_MODE, GATEWAY_AGENT_TYPE, GATEWAY_LISTEN_HOST, GATEWAY_PAC_ADVERTISE_ADDR, GATEWAY_AGENT_ID, GATEWAY_AGENT_HOSTNAME)
agent_identity.py	persistent agent ID + host/OS metadata for the fleet inventory
agent_heartbeat.py	periodic heartbeat, fleet config pull, remote-uninstall intent, health detail
agent_health.py	port checks (port_open, free_proxy_port), health loop
agent_pac_content.py	backend reachability check, PAC fetch, local PAC write
agent_pac_server.py	local PAC/health HTTP server (PAC_HTTP_PORT, default 18195, falls back to +1/+2 if busy)
agent_pac_orchestration.py	apply PAC with cache-bust, clear/restore runtime, fail-open direct mode
agent_proxy_engine.py	runs the mitmproxy worker child process
agent_proxy_bundle.py	SHA-tracked hot-update bundle, mark-bad, relaunch, running SHA
agent_certs.py	installs the mitmproxy CA into the OS trust store, writes ca_install_status.txt
agent_browser_policy.py	browser QUIC / DoH policy adjustments
agent_autoupdate.py	compares the packaged agent release metadata and handles approved update bundles when configured
agent_lifecycle.py	first-run prompt, uninstall flow (key-gated), user messages
agent_logging.py	file logging, get_resource_path
agent_http.py, agent_state.py, guard_platform.py	HTTP client, local state, OS paths (data_dir, log_hint_path, register_autostart)

Default paths: data `%LOCALAPPDATA%\YP-Raksha\Guard` (Windows) / `~/Library/Application Support/YP-Raksha/Guard` (macOS); log `gateway_guard.log`; CA status `ca_install_status.txt`.

mitmproxy addon

The addon is **not** a normal Python package. `gateway_proxy_parts/MANIFEST.txt` lists the parts that are loaded **in order into one shared namespace** (preserving the original single-file monolith behaviour):

```
config_caches_rules.py      config, caches, target/rule fetch + matching
helpers_prompts.py          prompt extraction + helper utilities
uploads_detect.py           upload / file detection
file_policy.py              per-file-type policy decisions
extract_office_backend.py   Office/PDF/image text extraction + backend calls
responses_inject.py         response rewriting / block injection (SSE aware)
responses_addon.py          response learning (e.g. filename discovery)
```

Flow: target-domain match → extract prompt/attachment → evaluate rules (regex locally; `ai_bot` via the configured LLM/Ollama) → verdict `Allowed | Redacted | Warned | Blocked | SiteBlocked | Bot Answered` → inject the browser-visible response (JSON error, Anthropic SSE Guard policy-block event, etc.) → POST the event to `/api/browser-ai/intercept`.

Editing any part requires rebuilding the Guard (`installer/build_installer.bat` on Windows, `installer/build_macos.sh` on macOS) and keeping cross-part names intact.

Guard distribution and hot update

- Build: `make build-guard-windows` (Inno Setup) / `make build-guard-mac` (+ `-pkg`) — see `apps/browser-guard/README.md`. Without `make`, run `installer\build_installer.bat`.
- Config sync: `python apps/browser-guard/scripts/sync_config_from_env.py` regenerates `config/gateway_guard_config.json` and the docs from `.env` (`SERVER_DOMAIN`, `GATEWAY_PROXY_ADDR`, `PAC_HTTP_PORT`, `GATEWAY_GUARD_SECRET`).
- Server-side artifacts live in `apps/browser-guard/release/` and are copied into the image (`/app/release`), so **Browser AI → Setup → Download Setup ZIP** serves them.
- `Browser AI → Setup → Rebuild & Publish` re-publishes the proxy/agent Python sources as a bundle (`/api/browser-ai/setup/proxy-bundle.zip`); installed agents pick it up via `agent_proxy_bundle.py`.
- Version file: `apps/browser-guard/release/VERSION.txt` → `1.1.30`.

Frontend

Build and integration

- Vite + React 19 + TanStack Router (file-based routes under `ui/app/**`, generated into `ui/app/routeTree.gen.ts`).
- State: Redux Toolkit with RTK Query APIs under `ui/app/_fallbacks/enterprise/lib/store/apis/*` (accessProfileApi, alertChannelsApi, auditLogsApi, businessUnitsApi, circuitBreakerApi, clusterApi, connectorsApi, loadBalancerApi, mcpToolGroupsApi, promptDeploymentsApi, rbacApi, scimApi, virtualKeyUsersApi, …).
- `npm run build:gateway` performs the product rebrand/enterprise build; `npm run build` runs Vite, typecheck and copies the embedded UI into `transports/gateway-http/ui`, which ships with the Gateway server; no separate production web server is required.
- Dev server: `npm run dev`; `make dev` runs UI + API together.
- Quality gates: `npm run typecheck` (tsc --noEmit), `npm run lint` (oxlint), `npm run format` (oxfmt).

Pages

Top level: `/` (landing), `/login`, `/signup`, `/oauth/consent`, `/pprof`, `/workspace/**`, plus `__root`, `__notFound`, `__error`.

Workspace pages (one `page.tsx` per route, each with a `layout.tsx`):

Section	Routes
Observability	`dashboard`, `logs`, `logs/connectors`, `mcp-logs`, `observability`, `audit-logs`, `alert-channels`
Browser AI	`browser-ai` (tabs: `overview`, `logs`, `search-logs`, `rules`, `targets`, `agents`, `telemetry`, `setup`)
Models	`providers`, `providers/model-limits`, `providers/routing-rules`, `model-catalog`, `model-limits`, `custom-pricing`, `custom-pricing/overrides`, `complexity-router`, `routing-rules`, `routing-rules/tree`, `circuit-breaker`
MCP Gateway	`mcp-gateway`, `mcp-registry`, `mcp-registry/library`, `mcp-registry/oauth-callback`, `mcp-tool-groups`, `mcp-sessions` (+ `auth`, `auth-failed`, `auth-success`), `oauth-grants`, `mcp-auth-config`, `mcp-settings`

Section	Routes
Governance	governance, governance/virtual-keys, governance/users, governance/teams, governance/business-units, governance/customers, governance/rbac, governance/access-profiles, rbac, virtual-keys, scim, scim/oauth-discover-callback
Guardrails	guardrails, guardrails/configuration, guardrails/providers
Platform	plugins, cluster, adaptive-routing (+ settings), prompt-repo (+ prompts), skills-repo, docs, config (+ api-keys, caching, client-settings, compatibility, feature-flags, logging, mcp-gateway, observability, performance-tuning, pricing-config, proxy, security)

Sidebar and section gating

ui/components/sidebar.tsx builds the menu from ui/lib/constants/workspaceSections.ts:

```
observability | browser-ai | models | mcp-gateway | plugins | governance |
guardrails | cluster-config | adaptive-routing | prompt-repository | skills-repository | settings
```

Each section has a defaultPath and optional child items. Visibility is decided by:

1. RBAC hooks — useRbac(RbacResource.X, RbacOperation.View).
2. Role checks for admin-only pages (Users requires admin or sub_admin).
3. Section grants for section-scoped sessions (scopedSidebarSections from authStatus.allowed_sections), with automatic redirect to getDefaultPathForSections(...) when the user lands on a route they are not granted.

Browser AI is reachable with Logs view RBAC **or** the admin/sub_admin role (hasBrowserAiAccess), matching the backend mapping of /api/browser-ai/* → Logs.

Browser AI UI implementation files

ui/app/workspace/browser-ai/: page.tsx (tabs, tables, dialogs), browserAiTypes.ts, browserAiConstants.ts (GUARD_BOT_OLLAMA_PROVIDER = "ollama", GUARD_BOT_OLLAMA_MODEL = "llama3.2", reference image max 512 KB), browserAiFormat.ts, browserAiLogHelpers.ts, browserAiGuardHelpers.ts, guardBotModelPickers.tsx, guardRuleAIEvaluatorFields.tsx, guardRuleImportDialog.tsx, targetImportDialog.tsx, regexLiveTestPanel.tsx, logPromptPreviewCell.tsx, logTimestampCell.tsx, logBadges.tsx, attachmentPreview.ts, relatedHosts.ts, plus import templates under examples/.

Types as implemented: GuardRuleAction = "BLOCK" | "REDACT" | "WARN", GuardRuleSeverity = "CRITICAL" | "HIGH" | "MEDIUM", GuardRuleEvalMode = "ai" | "regex".

Security model

Area	Implementation
Dashboard sessions	Session rows stored in DB; token cookie with HttpOnly + SameSite=Lax (+Secure only for real TLS or a trusted proxy sending X-Forwarded-Proto: https); default 24 h TTL
WebSocket auth	Cookie-only (/api/session/ws-ticket issues short-lived tickets); legacy ?token= query auth removed (token-leak risk)
Inference auth	Virtual Key or provider key via Authorization: Bearer / x-api-key; enforce_auth_on_inference flag
Brute force	Per-IP: 30 failed logins / 15 min window; per-username lockout rows in the DB; remaining minutes returned to the client

Area	Implementation
Password reset	`PASSWORD_RESET_SECRET` (min 32 chars) HMAC-signed token, 10 min expiry; 5 requests/hour/target + 10/hour/IP; OTP max 5 attempts; 2 min cooldown
Registration	Self-signup with e-mailed code (`email_unverified`) → admin approval (`pending` → `approved`) or `rejected`; SCIM can set `disabled`
Authorization	Two RBAC layers (resource×operation, section grants) + audit middleware recording mutating `/api/*` actions
Secrets at rest	`framework/encrypt` encryption for DB-stored secrets when `GATEWAY_ENCRYPTION_KEY` is set (provider keys, alert channels, uninstall keys)
Guard fleet	Shared `GATEWAY_GUARD_SECRET`, fail-closed with `GATEWAY_GUARD_REQUIRE_SECRET=1`; PAC hijack protection via `GATEWAY_PAC_ALLOW_QUERY_PROXY=0`
Guard uninstall	Company and per-agent uninstall keys, stored hashed + encrypted, rotatable, with an audit trail and remote-uninstall flow
Transport	TLS terminated at the edge/reverse proxy; `TRUST_PROXY_HEADERS=1` honours `X-Forwarded-Proto`/`X-Forwarded-For` **only** from trusted peers (`TRUSTED_PROXIES` required for public edges)
CORS	`client.allowed_origins` (from `SERVER_DOMAIN`); optional `ALLOW_LOCALHOST_CORS=1` for a local UI against a remote API
Headers	`SecurityHeadersMiddleware` applied to every response
Open bootstrap	When auth is disabled, unauthenticated admin is allowed only from loopback or with `ALLOW_OPEN_AUTH=1` (local bootstrap only)
Inference hardening	Provider headers normalised (`x-api-key` removed), password-as-Bearer removed, error sanitisation (`errorsanitizer.go`)
Destructive ops	Admin-only guards (`adminOnlyLogs`, `requireGuardAdmin`); attachment access privileged

Build test and developer workflow

Gateway

```
make setup-workspace     # go work use ./core ./framework ./plugins/... ./transports
make dev                 # UI (vite) + API with hot reload (air)
make build               # build UI then the Go binary
make run                 # build + run
make fmt ; make lint     # gofmt / linter

make test                # Gateway HTTP transport tests (gotestsum)
make test-core           # core provider tests (PROVIDER=openai TESTCASE=… PATTERN=…)
make test-framework      # framework tests
make test-plugins        # plugin tests
make test-http-transport # HTTP transport tests
make test-governance     # governance tests
make test-mcp            # MCP tests (TYPE=… TESTCASE=…)
make test-semantic-cache # semantic cache e2e
make test-all            # all of the above
make run-e2e ; make run-e2e-ui ; make run-e2e-headed ; make run-e2e-api
make test-integrations-py ; make test-integrations-ts
make run-provider-harness-test ; make run-cli-harness-test
```

Reports: `make generate-html-reports`; cleanup with `make clean-test-reports`.

UI

```
cd ui
npm ci
npm run dev            # Vite dev server
npm run typecheck      # tsc --noEmit
```

```
npm run lint            # oxlint
npm run build           # vite build + typecheck + copy into transports/gateway-http/ui
```

Guard

```
make sync-guard-config      # regenerate Guard config/docs from .env
make build-guard-windows    # Windows: Setup.exe + Windows ZIP (Inno Setup 6 required)
make build-guard-mac        # macOS: .app + macOS ZIP (must run on a Mac)
make build-guard-mac-pkg    # macOS: .pkg from the existing .app
make help-guard             # list Guard targets
```

Guard unit tests live in `apps/browser-guard/agent/tests/` and `apps/browser-guard/proxy/tests/` (PAC targets, proxy bundle, file-type extraction, latency, rule actions, target predict, upload names).

Docker Helm

```
make docker-image    # LOCAL=1 uses deploy/docker/Dockerfile.local (go workspace based)
make docker-run      # run the container (CONFIG=… optional)
make helm-index      # repackage chart + regenerate deploy/helm/index.yaml
```

Conventions

- Go modules resolve through `go.work`; add new modules with `go work use`.
- Handlers keep the `RegisterRoutes(router, middlewares...)` pattern and are grouped by feature.
- UI files use camelCase names, Radix primitives from `components/ui`, and RTK Query for API calls.
- Guard proxy parts must stay in `MANIFEST.txt` order.

Extension points

Goal	Where to work
Add an LLM provider	new package under `core/providers/<name>` implementing the provider contract, then register it in the provider registry
Add a pass-through API surface	new file in `transports/gateway-http/integrations/` + registration in `integrations/router.go`
Add a plugin	new module under `plugins/<name>` exposing `PluginName`; add a `case` in `loadBuiltinPlugin` (`server/plugins.go`) and add the module to `go.work`
Add a dashboard API	new `handlers/<feature>.go` with `RegisterRoutes`, then call it from `server.RegisterAPIRoutes`
Gate a new route with RBAC	extend `PathRequirementFor` (`framework/rbac/enforce.go`) and, for section-scoped sessions, `SectionRequirementFor` (`framework/rbac/sections.go`)
Add an RBAC resource	add it to `RBACResourceNames` (`framework/configstore/workspace.go`); permissions are generated from the cartesian catalog
Add a UI page	create `ui/app/workspace/<route>/page.tsx` + `layout.tsx`, add the section item in `ui/lib/constants/workspaceSections.ts`, and the nav entry in `ui/components/sidebar.tsx`
Add a Guard detection	edit the relevant part in `apps/browser-guard/proxy/gateway_proxy_parts/` (keep MANIFEST order) and rebuild the Guard
Add a Guard rule type	`BrowserGuardRule` (`framework/logstore/browser_ai.go`) + the UI rule editor + the proxy evaluator
Change gateway guardrail behaviour	`plugins/guardrails` + `/api/guardrails/*`

Glossary

Term	Meaning in this codebase
Virtual Key (VK)	A gateway-issued credential that maps to budgets/limits/teams/customers and is used by inference clients
ConfigStore	Config/workspace database (users, roles, keys, providers, settings)
LogsStore	Log database (LLM logs, MCP logs, Browser AI logs, aggregates)
Section grant	Sidebar-section permission (`governance/users`) used for section-scoped sessions
DAC	Data access scope on a role: `all-data` or `own-data`
Target website	A monitored AI domain (with platform name, host role and monitored/blocked flags)
Guard rule	A detection rule: `regex` (pattern) or `ai_bot` (LLM-judged, optional reference image)
Guard agent	A Guard installation reporting to the fleet (`endpoint` or `network`)
Fleet config	Company-wide Guard defaults that agents pull on heartbeat
PAC	Proxy auto-config script served by the backend, pointing browsers at the Guard proxy
Proxy bundle	Server-published hot-update bundle of the Guard proxy/agent Python code
Uninstall key	Company/per-agent secret required to turn the Guard off
Intercept	The Guard → backend event that records a prompt, upload or verdict

Enterprise deployment context

This manual describes implementation behavior and the repository architecture. It is not a substitute for a bank-specific design, a release support matrix, security certification, performance benchmark or operating procedure. The companion On-Premises Deployment Architecture provides a separately labelled planning envelope for DC, DR and UAT at 120,000 managed endpoints.

For strict residency, keep the Gateway, relational stores, Browser AI logs, model services, collectors, backups, DNS, identity, SMTP relay and artifact repositories under bank control in India. The product also contains provider and connector integration paths that can send data externally when configured. Disable these or route them to internal services, then enforce default-deny egress at the network layer. Review the architecture document's data-flow map and verification checklist before representing the deployment as no-egress.

The repository contains Windows and macOS Browser Guard packaging paths. It does not provide a complete versioned browser/VDI certification matrix in the source reviewed for this pack. Treat browser and VDI combinations as UAT targets until exact release-level compatibility evidence is recorded. Likewise, cluster functionality is not a cross-site DR guarantee; database replication, site promotion and failback are infrastructure and operations responsibilities.

Implementation boundaries relevant to product selection

Area	Repository evidence	Selection/deployment implication
Provider support	Provider adapter packages exist for multiple vendor families and operation types	Match required operations against the chosen adapter; do not assume equal feature coverage
Persistence	ConfigStore and LogsStore have PostgreSQL, MySQL and SQLite drivers	This pack uses PostgreSQL as the planning baseline; verify the release/database support matrix

Area	Repository evidence	Selection/deployment implication
Browser Guard endpoints	Windows and macOS packaging/build paths exist	Verify exact OS editions, versions, certificate deployment and endpoint management in UAT
Browser and VDI	Proxy/PAC and certificate mechanisms exist; a complete certification matrix is not present	Record exact tested browser/VDI versions; do not claim vendor certification without evidence
Data residency	Provider, telemetry and connector egress capabilities exist	Use internal endpoints and block public egress to establish the bank's residency boundary
DR	Application clustering and deployment tools exist	Design and exercise database/site replication, promotion, routing and failback separately
Performance	No benchmark result in this manual	Use the planning assumptions in the architecture document and qualify with a documented load test

Operational security review points

Before production, review the release-specific deployment with bank security and operations owners. At minimum, address privileged role provisioning, authentication/identity integration, secrets storage and rotation, database and backup encryption, TLS termination and trusted proxy configuration, audit access, payload logging, record retention, security alert routing, service account permissions, Guard agent update integrity, vulnerability scanning and emergency rollback.

Use internal artifact mirrors and review the actual release configuration. The repository's example deployment files contain public-service defaults; they are not evidence that a disconnected or no-egress deployment has been validated. The network owner should test the resulting system rather than relying on the intended application configuration.

Product architecture and request flows

YP-Raksha is organized as a Gateway service with an embedded administration UI, management APIs, inference APIs and optional integrations. The deployment has separate logical control and data planes, while model execution and selected storage/operations services can be separate bank-managed services.

Inference request lifecycle

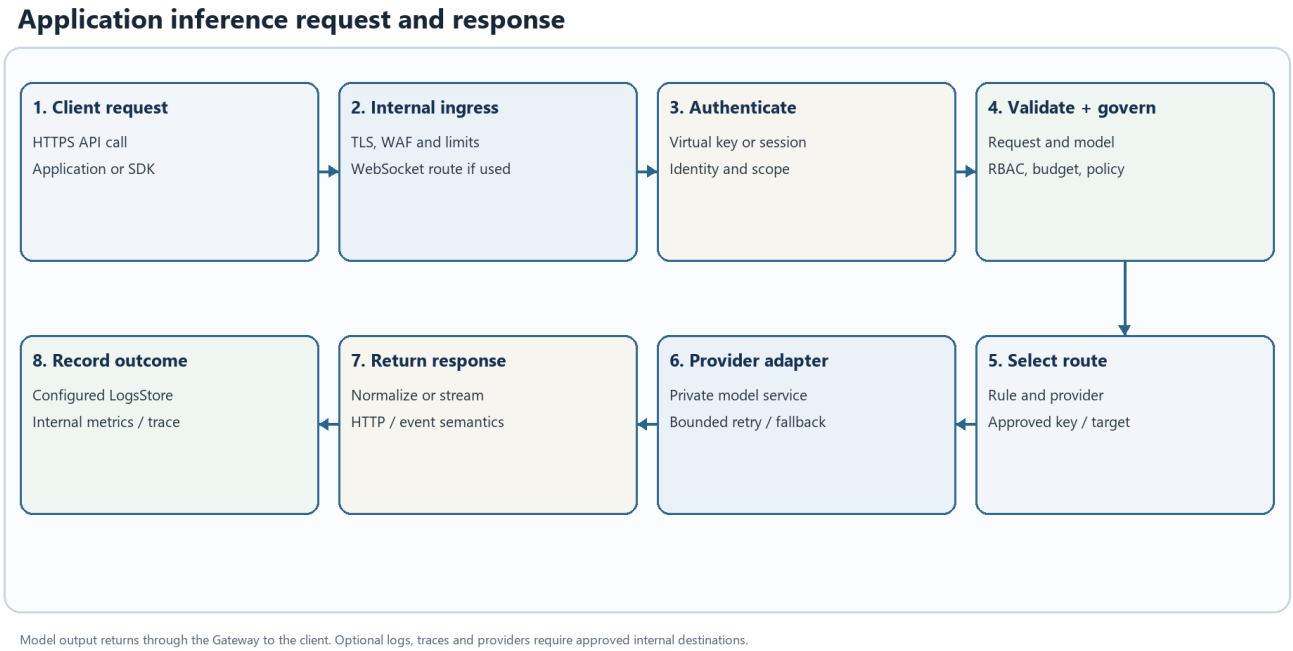


Figure 2. Application inference request and response. This is a logical workflow; enabled stages depend on deployment configuration.

1. The client submits an API request over HTTPS to the internal ingress. The route can be a standard inference endpoint or a supported provider/client-compatible integration.
2. The Gateway creates or accepts a request identifier, resolves credential/session context, authenticates the caller and applies configured virtual-key, role, budget and rate-limit checks.
3. Request validation and model/provider resolution run before provider execution. Missing or ambiguous provider/model data returns a client error; it is not treated as an upstream outage.
4. Policy hooks, compatibility conversion, routing rules, model selection and provider-key selection prepare the call. Enabled guardrail or cache plugins participate according to their configuration.
5. The selected provider adapter makes the outbound request to the configured destination. In a strict bank deployment this is a private bank-hosted or otherwise approved endpoint.
6. The core evaluates retry and key-rotation eligibility. If the call succeeds, the adapter normalizes the provider response or stream to the requested API shape.
7. Configured post-hooks run and response metadata is returned to the caller. Once a stream begins, a later failure cannot change the already-sent HTTP status.
8. Logging, tracing and metrics follow enabled configuration. Payload capture is separately controlled and must be minimized for sensitive data.

This is a logical sequence, not an unconditional feature promise. Provider/model support, API shapes, retry safety, guardrail failure policy and logging behavior are release- and configuration-specific.

Browser AI Guard decision lifecycle

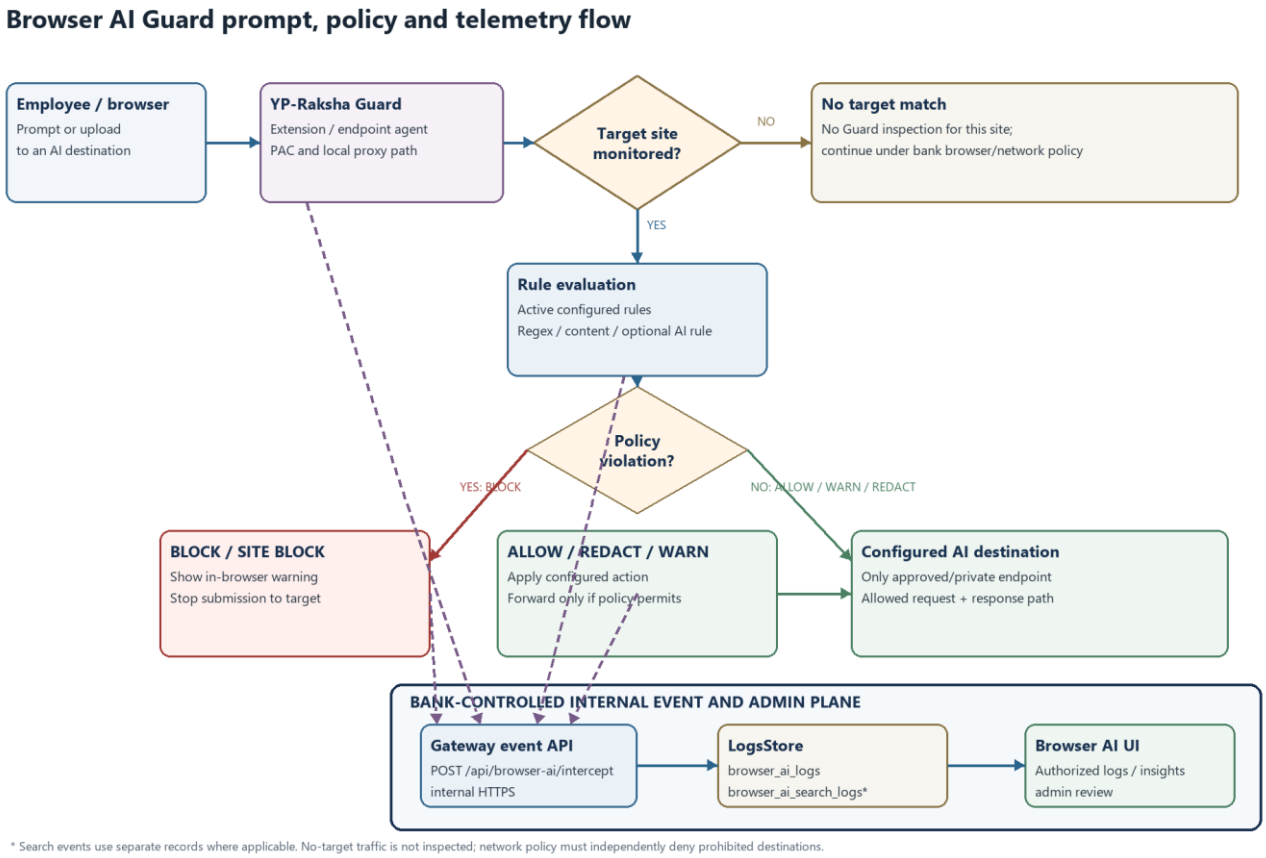


Figure 3. Browser AI Guard request, verdict and internal event flow. Exact behavior depends on configured rules and the validated release.

1. A managed browser requests a configured AI destination. Endpoint policy/PAC directs selected traffic to the local Guard proxy or an approved internal shared proxy.
2. The proxy checks target configuration, identifies the relevant request and extracts supported prompt or file content for policy evaluation.
3. Local regex rules are evaluated. If an AI Guard Bot rule is configured, the configured model/service may also be called; that destination must be approved and private for no-egress use.
4. The policy engine selects the configured action: allow, redact, warn, block, site-block or bot response. Matching order, rule scope and unmatched-site behavior must be acceptance-tested for the release.
5. Only traffic that policy allows to proceed is forwarded to the target. Redaction changes request content according to the configured rule; block/site-block prevents the upstream request. Warn behavior follows the configured control policy.
6. The response is passed back or transformed according to the proxy implementation. Streaming formats use provider-specific response handling; validate required providers and browser versions.
7. The Guard posts the policy event to the internal Browser AI API. The Gateway records it in LogsStore; permitted administrators review it in the embedded UI.
8. If the Gateway or event API is unreachable, do not assume this meets a bank fail-closed requirement. Combine endpoint settings with network rules that deny direct access to unapproved AI sites.

The exact content captured depends on traffic type and extraction support. Configured regex rules can express approved sensitive-term or pattern checks for DLP-style policy; the reviewed source does not establish a separate certified DLP engine. Treat prompts, file text, filenames, URLs and user identifiers as sensitive; configure collection, access, retention and deletion with the bank data owner.

Administration UI and policy change lifecycle

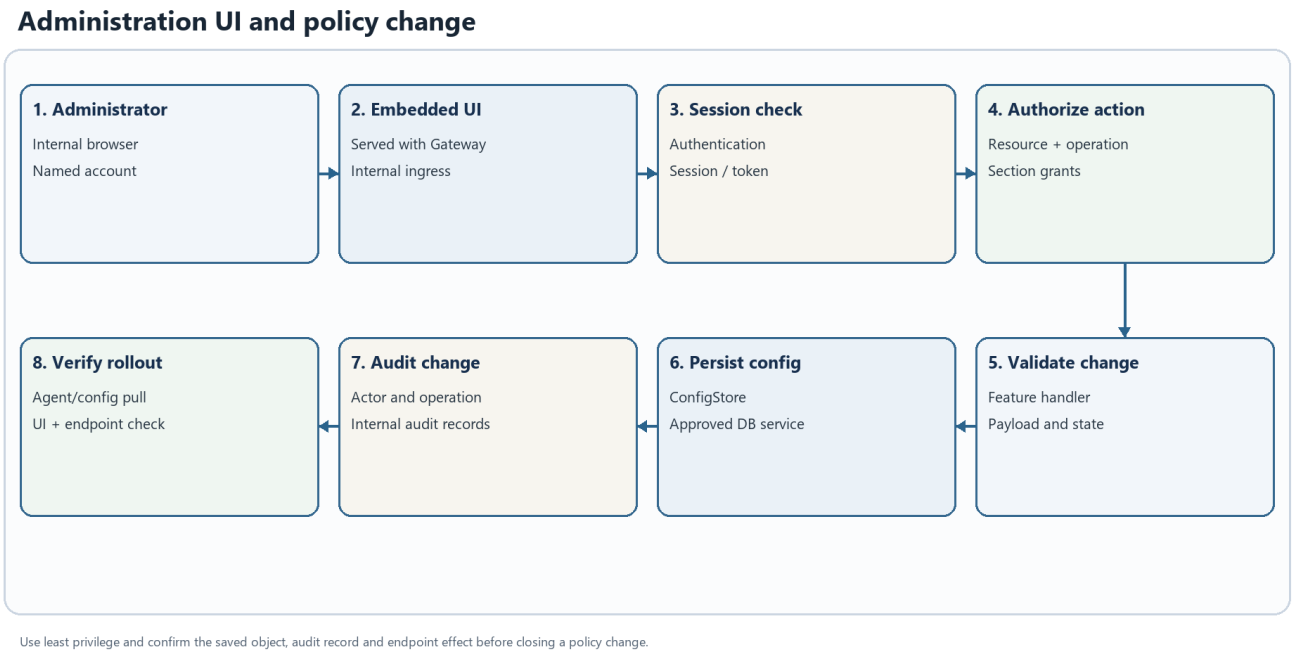


Figure 4. Administration UI and policy change. This is a logical workflow; enabled stages depend on deployment configuration.

1. An administrator opens the UI served with the Gateway application through the bank ingress; a separate public UI host is not required by the product build.
2. The administrator signs in through the configured authentication path. The browser holds a session credential according to application cookie/session configuration.
3. Each management API request passes authentication, audit middleware for mutating routes, resource-operation RBAC and applicable section-grant checks.
4. The feature handler validates the payload and requested operation. Invalid input returns a client error; a role denial returns an authorization error.
5. A valid update is written to the relevant ConfigStore model. If the action also changes grants or dependent objects, verify the full API result and audit event.
6. Mutating operations are added to the audit path. Operational logs and metrics are separate from business audit records and have separate access/retention needs.
7. Guard agents obtain fleet settings, rules and targets from internal Browser AI APIs during configured polling/heartbeat. Validate propagation time and endpoint identity in UAT.
8. The administrator verifies the resulting object, audit entry, service health and representative endpoint behavior.

Use named accounts and least privilege. The admin role is broad by design; scope other roles by resource, operation and section. Review configuration changes under the bank change process.

Optional MCP tool lifecycle

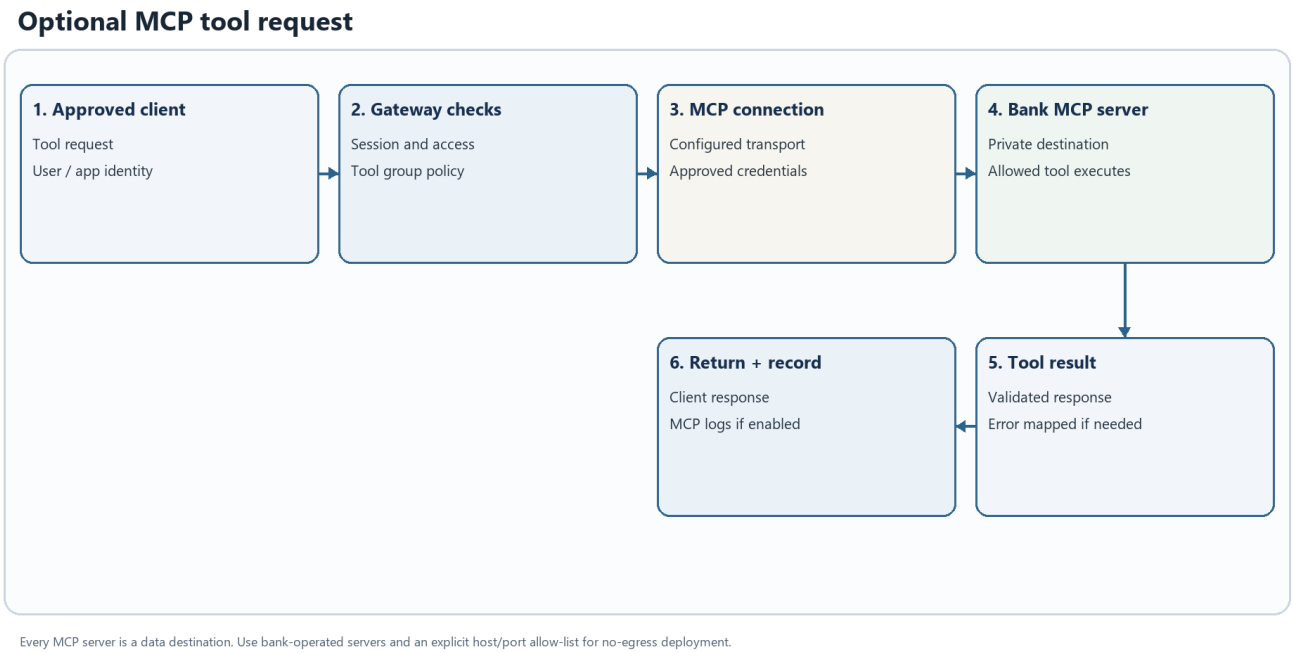


Figure 5. Optional MCP tool request. This is a logical workflow; enabled stages depend on deployment configuration.

1. An approved client request selects an enabled MCP client, session and tool group.
2. The Gateway checks the requesting identity, tool permissions and configured credentials.
3. The MCP client connects to the configured server using its selected transport and authentication method.
4. The remote tool executes and returns a result to the Gateway.
5. The Gateway returns the result to the client and records MCP usage/error information according to logging configuration.

Every MCP server is a potential data destination. For no-egress use, allow only bank-operated MCP servers, limit reachable hosts and ports, and review tool inputs, outputs, credentials and logs.

State scaling and component ownership

Component	Runtime responsibility	Availability and recovery responsibility
Gateway replicas	HTTP APIs, inference orchestration, policy integration, UI and optional background features	Platform team: replica placement, ingress, resource limits, rolling updates and health probes
ConfigStore	Users, roles, settings, providers, keys, routing and operational configuration	DBA/platform: HA, TLS, connection limits, backups, restore and site replication
LogsStore	Inference, audit, MCP, Browser AI event and aggregation records	DBA/data owner: retention, index maintenance, archive, access and recovery
Attachment and release files	Browser AI attachment bytes and Guard/package artifacts on configured paths	Platform team: persistent/shared volume behavior, permissions, capacity, backup and retention
Private model endpoints	Model execution and model-side queueing	Model owner: locality, authorization, capacity, quotas and availability
Internal observability	Metrics, application logs and trace receivers where enabled	Operations/security: internal-only endpoints, sensitive-label review, retention and alerts

Component	Runtime responsibility	Availability and recovery responsibility
Guard fleet	Endpoint proxy/PAC, local health, configuration exchange and event delivery	Endpoint/network teams: rollout, certificates, local data path, proxy policy, update and recovery

The relational database supports several drivers in the implementation. PostgreSQL is the reference production planning choice in this pack; qualify a release/database combination with the product owner and bank DBA. Optional cache, vector and object services are not all required for every deployment.

Error handling retries and recovery

Error response contract

The HTTP layer returns a JSON Gateway error object for the standard API error path. Fields are optional by error class; this is an illustrative shape, not a fixed response for every route.

```
{
  "is_gateway_error": false,
  "status_code": 429,
  "error": {
    "type": "rate_limit_error",
    "code": "rate_limit_error",
    "message": "Request limit reached",
    "event_id": "optional-event-id"
  },
  "extra_fields": {
    "provider": "configured-provider",
    "request_type": "configured-operation"
  }
}
```

An incoming x-request-id can be used as the request identifier; if absent the request context creates one. Preserve the identifier, timestamp, route, deployment/site, provider/model label, HTTP status, retry count and client application name when opening an incident. Do not include prompts, credentials or full payloads in an incident ticket unless approved.

The HTTP error path sanitizes selected internal error details before returning them to the client. Stack traces, SQL/database diagnostics and HTML/status-page bodies are hidden behind a client-safe internal message. Operational logs remain the diagnostic source; restrict them because they may contain sensitive request context.

Common failure classes and caller action

HTTP status / class	Typical meaning	Caller response	Operator check
400 Bad Request	Invalid JSON/parameters, missing provider/model, unsupported operation/stream combination	Correct the request; do not retry unchanged input	Validate schema, route and model selection
401 Unauthorized	Missing/invalid client credential, expired session, or provider credential error	Re-authenticate or use a valid virtual key; do not retry a known bad key indefinitely	Check caller identity, key state, provider credential and session configuration
403 Forbidden	Caller lacks application/role permission, or provider key/account is denied	Request the required grant or use an authorized account	Inspect RBAC/section grants, virtual-key scope and provider permissions
404 Not Found	Unknown Gateway route/resource, model or object identifier	Verify endpoint, ID and API version	Confirm route registration, object lifecycle and model catalog
409 Conflict	Resource/state conflict on operations that use conflict semantics	Refresh resource state and retry only after resolving conflict	Inspect concurrent changes, duplicate operation and audit event

HTTP status / class	Typical meaning	Caller response	Operator check
429 Too Many Requests	Gateway quota/rate limit, Guard uninstall lockout, or provider throttling	Back off; honor returned retry guidance; avoid burst retry loops	Separate caller budget from provider quota and inspect key attempts
500 Internal Server Error	Application, persistence or feature-handler failure	Retry only if operation is safe and cause is transient; retain request ID	Check service logs, DB connectivity, schema/configuration and plugin startup
502 Bad Gateway	Provider/network failure, exhausted upstream credentials, or upstream response failure	Use bounded retry only where request semantics permit; do not assume write completion	Check private endpoint, TLS, route, provider status and key pool
503 Service Unavailable	Required store/service unavailable, no eligible key, Guard secret missing or health dependency failure	Retry after recovery with jitter; do not bypass policy	Check readiness dependencies, DB/vector store, secret injection and route eligibility
504 Gateway Timeout / deadline	Upstream or downstream operation exceeded its configured deadline	Treat write/submit completion as unknown until reconciled	Check request/ingress timeout, provider latency and DB wait
WebSocket/realtime error event	Route-specific handshake, session or provider event failure	Follow event type/code and approved reconnect policy	Check ingress upgrade headers, session/ticket, allowed origins and provider support

This is a diagnostic map, not a universal one-to-one status contract. Handlers and provider adapters can return different codes/messages; inspect the selected release API schema and acceptance results before client integration.

Retry key rotation and fallback behavior

The core retry loop uses configured MaxRetries and network backoff values. Backoff is exponential with jitter and bounded by its configured maximum. Current source classifies failures as follows:

Failure	Core behavior	Client/application design
Network/request transport error	Eligible for bounded retry; retain the same key because a new credential does not repair the path	Set finite timeouts; prevent caller and Gateway retries from multiplying without limit
Upstream 500, 502, 503 or 504	Eligible for bounded retry using the same key	Ensure the total deadline covers attempts and backoff
Upstream 401, 402 or 403	Eligible for key rotation when a provider key pool exists; failed credential is marked dead for that request	Correct or remove invalid keys; monitor credential exhaustion
Upstream 429 / rate-limit signal	Eligible for retry and key rotation where possible; a rate-limited key may be reconsidered after pool exhaustion	Respect quotas; avoid synchronized retries across replicas
Request-bound 4xx such as invalid input	Not retried by the core retry classifier	Fix the request; changing keys does not correct malformed input
Gateway internal error / request cancellation	Retry loop stops	Diagnose locally or cancel; do not automatically replay non-idempotent calls
Streaming error found before first usable chunk	Can be classified and retried before a stream is returned	Validate per provider; never assume a retry after content has been sent
Configured circuit breaker / fallback	Separate routing control can select an approved fallback when policy conditions match	Fallback is not automatic for every error; keep targets within approved locality/data class

A transport timeout can happen after an upstream service accepted the request. For non-idempotent operations such as file, image, video, batch or tool creation, completion may be unknown. Use operation IDs, provider status lookup,

deduplication/idempotency keys where supported, and reconciliation before resubmission. The Gateway does not make every operation exactly-once.

Streaming WebSocket and partial response errors

Before headers/body are committed, a handler can return a normal HTTP error. After SSE/WebSocket/realtime output begins, the HTTP status cannot be replaced. A provider failure mid-stream can appear as a protocol error event, stream termination or incomplete response. Clients should:

- treat an incomplete stream as failed, retain the request ID and stop presenting partial output as a completed answer;
- close the stream and follow a bounded, application-approved reconnect policy;
- avoid replaying side-effecting tool calls without checking provider/tool state;
- align ingress idle/read timeouts with approved stream lifetime;
- verify SSE, WebSocket and realtime behavior separately in UAT for every required provider.

Operational recovery matrix

Symptom	Likely failure domain	First checks	Recovery path
/health returns 503	ConfigStore, LogsStore or configured VectorStore ping failure	Service DNS, port, TLS, credentials, pool exhaustion and DB status	Restore dependency; verify health and a read-only API; do not substitute disabled checks for recovery
All inference returns 400	Client request or model/provider resolution	Request schema, model name, provider config and operation	Correct payload or add the approved model/provider route
Inference returns 502/503	Provider endpoint, credentials, route policy or no eligible provider key	Gateway DNS/TLS/connectivity, provider health, key pool, virtual-key grants and egress denies	Restore private route or approved key; reduce traffic or use a pre-approved fallback
Repeated 429s	Caller quota, budget/rate policy or provider quota	Compare Gateway usage limits with provider response and per-key attempt trail	Apply bounded jitter, rebalance approved keys or request capacity; do not raise quota without owner approval
Guard heartbeat rejected or 503	Guard secret absent/mismatched, wrong backend URL, certificate or route	GATEWAY_GUARD_SECRET, GATEWAY_GUARD_REQUIRE_SECRET, agent version, URL, DNS/TLS and firewall	Correct via approved secrets and endpoint rollout; rotate if exposed
Prompt is not blocked	PAC bypass, target mismatch, rule scope/order or unsupported extraction	PAC result, active target, rule status, agent health, local proxy logs and direct egress path	Correct target/rule and retest; deny public AI at network edge for protected users
Browser works but event is missing	Event API reachability, secret, LogsStore failure or retention/filter	Agent delivery status, health, Browser AI API and time/filter	Restore internal API/store connectivity; confirm ingest and retention result
Uploaded attachment unavailable	Attachment path permission, expiry, volume or metadata/DB mismatch	Persistent path, free space, owner/mode, expiry and pod placement	Restore approved persistent storage; reconcile expired/orphaned metadata; retest access
Admin update returns 403	Session, role/resource permission or section grant	Identity, role, route requirement and audit log	Grant least-privilege access through authorized administrator
UI works but APIs fail	Ingress rewrite, cookie/origin, session expiry or API health	Browser network response, ingress route, TLS/cookies and Gateway log	Correct routing/session policy; verify a read-only API before retrying a mutation
WebSocket/realtime handshake fails	Ingress upgrade, short-lived ticket, auth/origin or provider support	Upgrade forwarding, ticket freshness, origin, route and provider	Fix ingress/auth and validate with a new ticket

Symptom	Likely failure domain	First checks	Recovery path
500 after configuration change	Invalid dependent config, DB write/migration or plugin issue	Request ID, application log, audit entry, DB health and startup/plugin status	Roll back the specific change via approved method; preserve audit history
No metrics/traces	Scrape/token, plugin/exporter or network rule	Metrics reachability, GATEWAY_METRICS_TOKEN, internal collector and egress policy	Correct internal scrape/export; do not enable a public collector as workaround

Incident evidence and escalation

Capture UTC timestamp; deployment/site and release digest; request ID or agent ID; route and operation; status/event type; provider/model label without credentials; retry count and attempt outcome; health/dependency status; matching log/trace IDs; recent approved change; and sanitized network error. Protect raw request/response content and attachments under bank data policy. Escalate product defects with a minimal reproducible request using synthetic content and exact release identifier.

Deployment prerequisites and configuration

Required platform dependencies

Dependency	Required planning decision
Runtime platform	Select bank container/Kubernetes version, node OS/CPU architecture, ingress class and support policy; verify the release image
Internal artifact supply chain	Mirror approved signed Gateway image, Guard installer/bundle, chart and model/config assets; scan and record digests
Database	Provide HA relational storage supported by the release; use PostgreSQL as the reference baseline, then validate version, TLS, pool, backup and migration
Storage	Provide durable capacity for database, configured attachment directory and generated/release artifacts that must survive pod replacement
Network	Provide internal DNS/NTP, ingress, private model routes, database routes, endpoint HTTPS, observability and encrypted DC-to-DR replication
Identity/certificates	Decide authentication, SSO/SCIM/AD if used, TLS termination, trusted proxy CIDRs and CA trust distribution
Secrets	Use bank secrets manager or encrypted Kubernetes Secrets; define rotation/recovery for DB, encryption, bootstrap admin and Guard secrets
Operations	Define internal logs/metrics/traces, metrics access, alert routes, retention and incident owner
Endpoint rollout	Define Windows/macOS package deployment, certificate/PAC policy, agent identity persistence, VDI treatment and rollback
Residency	Approve provider/model, SMTP, licensing, update, connector and telemetry destinations; deny all other egress at network layer

Runtime configuration groups

The source .env.example is a template with development-oriented values. Inject production secrets through the approved secret manager.

Configuration group	Representative current keys	Purpose and handling
HTTP/runtime	APP_PORT, APP_HOST, APP_DIR, LOG_LEVEL, LOG_STYLE	Bind intended interfaces; persist APP_DIR data where local attachments/artifacts must survive
Database	DB_TYPE, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_SSL_MODE	Separate credentials by environment; require database TLS where supported
Secret encryption	GATEWAY_ENCRYPTION_KEY	Keep stable and protected under dual control; losing it may make encrypted stored credentials unrecoverable
Guard fleet	GATEWAY_GUARD_SECRET, GATEWAY_GUARD_REQUIRE_SECRET, GATEWAY_BACKEND_URL, GATEWAY_PROXY_ADDR, GATEWAY_AGENT_TYPE	Require strong secret in production; distribute securely and rotate through an endpoint rollout
PAC behavior	GATEWAY_PAC_ALLOW_QUERY_PROXY, GATEWAY_PAC_URL, GATEWAY_PAC_SYNC_SECONDS, GATEWAY_PAC_ADVERTISE_ADDR	Keep client override disabled unless approved; validate proxy address from endpoint network
Admin and mail	ADMIN_EMAIL, ADMIN_PASSWORD, SMTP_*, PASSWORD_RESET_SECRET	Change bootstrap credentials; use internal relay or disable mail; protect reset signing secret
Metrics/proxy trust	GATEWAY_METRICS_TOKEN, TRUST_PROXY_HEADERS, TRUSTED_PROXIES	Restrict metrics to internal scrapers; trust forwarded headers only from ingress peers
Branding	GATEWAY_COMPANY_NAME, GATEWAY_COMPANY_LOGO and product/footer settings	Set approved company/product/footer copy; validate landing, login and footer before release

Use only names present in the selected release's .env.example, chart and configuration loader. Do not copy sample passwords, public SMTP/DNS or public URLs into production.

Production packaging cautions

- Root Compose is a development/evaluation starting point. The current example includes public DNS resolvers and runs the application as root. Do not deploy it unchanged for bank production/no-egress.
- The optional Compose network-proxy profile is a lab configuration; permissive interception/TLS settings are not a production baseline.
- The Helm chart and values include public image/catalog examples. Confirm the chart version maps to the approved YP-Raksha release, mirror it internally, pin immutable digests and remove public endpoint defaults.
- Supply credentials through secret references or the bank secret integration. Do not put cleartext credentials in values files, environment literals, command lines or tickets.
- Use internal TLS ingress, database endpoints, signed artifacts and restricted service accounts. Keep profiling and development routes unavailable in production.
- UAT requires its own database, secrets, admin identities, model endpoint and telemetry. Do not restore production records into UAT without approved masking.

Installation upgrade and rollback runbook

1. **Approve the design.** Confirm release/version, artifact digest, runtime support, OS/browser/VDI targets, network flows, data classification, retention, RPO/RTO and owners.
2. **Prepare internal dependencies.** Mirror approved images, charts, Guard packages and static assets. Confirm DNS, NTP, certificates, database, private model and collectors from the Gateway network.
3. **Create secrets/database.** Provision environment-specific DB credentials, encryption key, Guard secret, reset secret, bootstrap admin and TLS materials. Apply least privilege and recovery controls.

4. **Prepare persistence.** Confirm ConfigStore/LogsStore schema migration, attachment ownership/capacity, DB pool limits, backup and site replication.
5. **Install to UAT first.** Use the approved bank manifest, internal registry and secret references. Disable external providers/connectors/exporters until approved.
6. **Run readiness/integration checks.** Check /health and /api/health, database read/write, authenticated UI, private inference, Guard heartbeat/config, synthetic policy event, internal metrics and denied public egress.
7. **Qualify acceptance and load.** Use synthetic prompts and representative payloads; test quota/rate, retry/fallback, streaming interruption, DB outage, restart, upgrade and restore. Record test conditions and results.
8. **Promote the same immutable artifact.** Move tested image/chart digest and approved configuration through change control. Do not rebuild or pull public registries separately in production.
9. **Canary and scale.** Roll out a small endpoint/application cohort, observe service/DB/Guard metrics, then expand by approved stages.
10. **Rollback safely.** Stop rollout, route traffic to previous approved version, restore compatible configuration and follow the database rollback plan. Never assume schema downgrade is safe; verify a pre-upgrade backup.
11. **Close the change.** Record versions/digests, approvals, test evidence, capacity, support contact, restore point, outstanding risks and review date.

Exact Helm values, replica counts and commands depend on bank platform and release chart. Use a validated release manifest rather than copying development examples.

Day 2 operations availability and data protection

Health and monitoring

GET /health and GET /api/health ping configured ConfigStore, LogsStore and VectorStore dependencies with a bounded timeout and can return 503 when a dependency check fails. A configuration option can disable these pings; if used, add separate dependency readiness checks and do not interpret process-only health as full service readiness. Use distinct liveness/readiness probes where supported.

Restrict /metrics to internal scrapers and protect it with the configured metrics token when enabled. Production profiling routes must remain disabled. Monitor request rate/latency/status by route/provider/model, retry/key-rotation counts, stream interruptions, DB pool/write pressure, migration status, storage growth, Guard heartbeat age, event delivery, agent versions, egress-deny events, backup status and DR replication lag.

Backup restore and DR dependencies

Back up relational records and durable attachment/package paths as a coordinated service set. Protect the encryption key and secret-recovery metadata using the bank's separate key process; a DB restore without compatible decryption material can make stored credentials unusable. Verify point-in-time recovery, attachment-to-metadata consistency, restore permissions, retention expiry and package provenance. Conduct an isolated restore before counting a backup as valid.

For DC/DR, database replication, DNS/load-balancer cutover, Gateway capacity, model endpoint availability, TLS, Guard secret, encryption key, internal telemetry and endpoint routing must all be in the exercise. Application cluster routing does not perform those site-level tasks by itself.

Change control and key rotation

Use an approved change record for provider, routing, guardrail, retention, PAC, Guard fleet and database changes. Record protected pre-change values, verify behavior with synthetic requests and retain the audit record. Rotate provider credentials by staged addition, validation, switch and revocation. Rotate the Guard secret with coordinated package/config rollout; confirm fail-closed behavior on protected APIs.

Capacity qualification

The 120,000-endpoint DC/DR/UAT figures in the companion architecture are planning estimates from declared activity assumptions, not a measured product limit or performance guarantee. Recalculate with representative request sizes, stream concurrency, Guard event volume, DB retention/index overhead, attachments and model latency. Increase replicas only after confirming database connections, durable storage and downstream model capacity.

Technical acceptance checklist

- [ ] Release identity, image digest, chart mapping and dependency versions are recorded.
- [ ] Application, endpoint, model, storage, telemetry and update flows are approved.
- [ ] Network egress is default-deny; public model/telemetry/analytics paths are blocked and deny logs stay internal.
- [ ] DB TLS, credentials, migrations, HA, backup and restore are demonstrated.
- [ ] Encryption key and Guard secret are managed, recoverable/rotatable and never embedded in client-visible configuration.
- [ ] RBAC/section grants are reviewed with test accounts; admin use is limited and audited.
- [ ] API error/retry behavior and streaming interruption are tested with synthetic content.
- [ ] Guard outcomes (allow, redact, warn, block, site-block) and bypass prevention are validated on every target.
- [ ] Attachment paths, retention, deletion and backup/restore are verified.
- [ ] Health, dependency readiness, internal metrics and alerts are tested; profiling is not exposed.
- [ ] DR failover/failback is exercised with measured RPO/RTO, secrets, DNS, model and collectors.
- [ ] Support and operations teams have troubleshooting, escalation and rollback procedures.

Source and maintenance note

This YP-Raksha technical manual is derived from the repository implementation, including `core/`, `framework/`, `transports/gateway-http/`, `apps/browser-guard/`, `ui/`, `deploy/` and the accompanying server and administrator guides. Refresh the route inventory, provider matrix, dependency versions, security behavior and deployment references whenever the product release changes. Record a release identifier and document owner in the bank’s controlled copy.